1. Explain ASP.NET Core

ASP.NET Core is a **fast, open-source, and cross-platform** framework by Microsoft used to build **web apps, APIs, and microservices** that run on Windows, Linux, and macOS.

Key Features of ASP.NET Core

1. **Cross-Platform** – Works on Windows, Linux, and macOS.
2. **High Performance** – Runs faster with less memory usage.
3. **Modular & Lightweight** – Uses only required components, making it efficient.
4. **Unified Framework** – Supports MVC, Web APIs, Blazor, and Razor Pages.
5. **Middleware Pipeline** – Handles requests efficiently using middleware.
6. **Built-in Dependency Injection** – Manages app services easily.
7. **Cloud & Microservices Ready** – Ideal for cloud apps and microservices.
8. **Security** – Supports authentication like JWT, OAuth, and Identity.
9. **Open-Source & Community-Driven** – Updated frequently with community support.

2. Explain the key differences between ASP.NET Core and ASP.Net.

Here are the key differences between **ASP.NET Core** and **ASP.NET** in simple points:

1. **Cross-Platform**: ASP.NET Core runs on Windows, Linux, and macOS, while ASP.NET works only on Windows.
2. **Performance**: ASP.NET Core is faster and more lightweight compared to ASP.NET.
3. **Modularity**: ASP.NET Core allows modular development with minimal dependencies, whereas ASP.NET has a heavier framework.
4. **Open-Source**: ASP.NET Core is open-source, while ASP.NET is closed-source and part of the .NET Framework.
5. **Hosting**: ASP.NET Core can be hosted on Kestrel, IIS, or even in Docker containers, whereas ASP.NET is limited to IIS.
6. **Architecture**: ASP.NET Core supports microservices and cloud-based development better than ASP.NET.
7. **Dependency Injection**: ASP.NET Core has built-in dependency injection, while ASP.NET relies on third-party libraries.
8. **Support & Future**: ASP.NET Core is the future of .NET development, while ASP.NET is legacy and no longer actively developed.

3.What are the main components of dot net core

The main components of **.NET Core** are:

1. **Runtime** – Executes applications and provides essential services like garbage collection and type safety.
2. **Base Class Library (BCL)** – A set of libraries providing basic functionalities like file handling, collections, and threading.
3. **SDK (Software Development Kit)** – Includes tools, compilers, and libraries needed for development.
4. **CLI (Command-Line Interface)** – A tool for building, running, and publishing .NET Core applications using commands.
5. **ASP.NET Core** – A framework for building web applications and APIs.
6. **Entity Framework Core (EF Core)** – A lightweight, cross-platform ORM for database operations.
7. **Cross-Platform Support** – Works on Windows, macOS, and Linux.

4. What are some characteristics of dot net core?

Here are some key characteristics of .NET Core in simple points:

1. **Cross-Platform** – Works on Windows, Linux, and macOS.
2. **Open-Source** – Free to use and modify.
3. **High Performance** – Faster than traditional .NET Framework.
4. **Modular** – Uses NuGet packages for flexibility.
5. **Microservices-Friendly** – Ideal for cloud-based and microservices applications.
6. **Lightweight** – Requires fewer system resources.
7. **Supports Multiple Languages** – Works with C#, F#, and VB.NET.
8. **Built for Modern Apps** – Great for web, mobile, and cloud apps.
9. **Easy Deployment** – Can be deployed as self-contained or framework-dependent.
10. **Active Community & Support** – Backed by Microsoft and developers worldwide.

5. Latest version of dot net core?

The latest version of .NET Core is 3.1, which reached end of support on December 13, 2022. Starting with .NET 5, Microsoft unified the .NET ecosystem, dropping the "Core" branding. The most recent release in this unified platform is .NET 8, which was released on November 14, 2023, and will be supported until November 10, 2026.

6. How is .NET Core SDK different from .NET Core Runtime?

Difference Between .NET Core SDK and .NET Core Runtime

1. **.NET Core SDK (Software Development Kit)**
   - Used for **developing and building** applications.
   - Includes **compilers, CLI tools, and libraries** for development.
   - Needed if you want to **write and compile** code.
   - Comes with the .NET Core **Runtime** included.
2. **.NET Core Runtime**
   - Used for **running** applications, not developing them.
   - Includes only the **libraries and components** required to execute a .NET Core app.
   - Does **not** have compilers or development tools.
   - Needed if you just want to **run an already built** application.

7. Explain Docker in .NET Core

What is Docker?

- A tool that packages applications and dependencies into lightweight containers.

Why Use Docker in .NET Core?

- Ensures the app runs the same on any system.
- Makes deployment easy and efficient.

How Docker Works in .NET Core?

- A .NET Core app is packaged into a **Docker image**.
- The image is used to create a **Docker container** (a running instance).
- The container runs the app in an isolated environment.

Basic Steps to Use Docker with .NET Core

- Install **Docker Desktop**.
- Create a .NET Core app using dotnet new webapi.
- Add a **Docker file** (instructions for building the image).
- Build and run the container using docker build and docker run.

Docker file Example for .NET Core App

dockerfile
CopyEdit
FROM mcr.microsoft.com/dotnet/aspnet:7.0
WORKDIR /app
COPY . .
ENTRYPOINT ["dotnet", "MyApp.dll"]

- This pulls a .NET runtime image, copies the app, and runs it.

Running a .NET Core App in Docker

- Build the image: docker build -t myapp .
- Run the container: docker run -p 5000:80 myapp
- Open http://localhost:5000 in a browser.

Benefits of Using Docker in .NET Core
✔ Works the same on all machines
✔ Easy to deploy anywhere
✔ Supports microservices and cloud hosting
✔ Reduces dependency issues

8. Share specific features of .NET Core?

Here are some key features of **.NET Core** in simple points:

1. **Cross-Platform** – Runs on Windows, Linux, and macOS.
2. **Open-Source** – Free and open-source framework by Microsoft.
3. **High Performance** – Optimized for speed and scalability.
4. **Modular & Lightweight** – Uses NuGet packages for only needed components.
5. **Microservices Support** – Ideal for building modern cloud-based applications.
6. **Docker & Kubernetes Friendly** – Easily containerized and deployable.
7. **Unified Platform** – Supports web, desktop, mobile, cloud, IoT, and AI.
8. **Dependency Injection** – Built-in DI support for better code management.
9. **Command-Line Interface (CLI)** – Can be developed and run from the command line.
10. **Side-by-Side Versioning** – Multiple versions can run on the same machine.

NuGet package

A **NuGet package** in .NET Core is a library or tool that you can add to your project to use extra features.

You can install it using the command: dotnet add package Package Name

Entity Framework Core (EF Core) in .NET Core

Entity Framework Core (EF Core) is Microsoft's lightweight, extensible, and cross-platform Object-Relational Mapper (ORM) for .NET Core applications. It simplifies database access by allowing developers to work with databases using .NET objects instead of SQL queries.

Key Features of EF Core

1. **Cross-Platform Support** - Works with .NET Core on Windows, Linux, and macOS.
2. **Code-First Approach** - Define models in C# and generate database schema.
3. **Database-First Approach** - Scaffold models from an existing database.
4. **Migrations** - Easily update database schema using dotnet ef migrations commands.
5. **LINQ Queries** - Use LINQ to interact with the database.
6. **Lazy & Eager Loading** - Fetch related data as needed.
7. **Multiple Database Providers** - Supports SQL Server, PostgreSQL, MySQL, SQLite, and more.

command line and command prompt

Command Line (CLI) in .NET Core

- The **Command Line Interface (CLI)** is used to run .NET Core applications, execute commands, and manage projects.
- The main tool is the dotnet command.

Command Prompt (cmd.exe) in .NET Core

- The **Command Prompt** (cmd) is the Windows terminal where you can run commands.
- You can execute cmd commands from a .NET Core application using System.Diagnostics.Process.

Running PowerShell or Bash from .NET Core

- Instead of cmd.exe, you can run **PowerShell** (powershell.exe) or **Bash** (/bin/bash).

Building CLI Applications in .NET Core

- You can create command-line applications using the System.CommandLine library.
- Install it:

```
CopyEdit
dotnet add package System.CommandLine
```

Summary

- **Command Line** (CLI) in .NET Core is used for managing and running .NET apps (dotnet commands).
- **Command Prompt** (cmd.exe) can be used to execute Windows commands from .NET Core.
- **PowerShell or Bash** can also be executed from a .NET Core app.
- **System.CommandLine** helps build interactive CLI applications in .NET Core.